

LAB REPORT

Lab Title	Dynamic Data Structures	Lab No.	12
Student Name	Li Rongyao	Student ID	2024103975
Date	2025/5/19		

Lab description/objectives:

1. Demonstrate understanding of the usage of dynamic data structures especially the link-list-based stack in C programming.
2. Implement the push, pop, sum functions as defined in the lab tasks/requirements.
3. Analyze the performance of the stack and each function.

Source code:

```
#include <stdio.h>
struct menu
{
    int num;
    struct menu *priorAddr;
};
struct menu *tosp = NULL;
void pushstk();
void poponestk();
void sumstk();
void popstk();
int main()
{
    int opt;
    printf("Option 1: Push Integer to Stack.\n");
    printf("Option 2: Pop Integer from Stack.\n");
    printf("Option 3: Calculate the sum of Stack Integers.\n");
    printf("Option 4: Quit.\n");
    do {
        printf("Enter the number to do corresponding task.\n");
        scanf("%d", &opt);
        switch (opt)
        {
            case 1:
                pushstk();
                break;
            case 2:
                poponestk();
                break;
```

```
        case 3:
            sumstk();
            break;
        default:
            popstk();
            break;
    }
} while (opt != 4);
return 0;
}

void pushstk()
{
    int tmp;
    printf("Please enter the integer you want to push.\n");
    scanf("%d", &tmp);
    struct menu* node = (struct menu *)malloc(sizeof(*tosp));
    if (node == NULL){
        printf("Failed to get the memory.\n");
        return;
    }
    node->num = tmp;
    node->priorAddr = tosp;
    tosp = node;
}

void poponestk()
{
    if (tosp != NULL)
    {
        int tmp = tosp->num;
        struct menu *tmpaddr = tosp->priorAddr;
        free(tosp);
        tosp = tmpaddr;
        printf("The integer %d on the top has been popped.\n", tmp);
    } else {
        printf("Error. The stack is empty.\n");
    }
}

void sumstk(){
    if (tosp == NULL){
        printf("Error. The stack is empty.\n");
        return;
    }
    int sum = 0;
```

```
struct menu *tmpptr = tosp;
while (tmpptr != NULL){
    sum += tmpptr->num;
    tmpptr = tmpptr->priorAddr;
}
printf("The sum of all integers on the stack is %d.\n", sum);
}
void popstk(){
    while (tosp != NULL){
        struct menu *tmpptr = tosp->priorAddr;
        free(tosp);
        tosp = tmpptr;
    }
    printf("The stack has been popped.\n");
}
```

Program output:

```
Option 1: Push Integer to Stack.
Option 2: Pop Integer from Stack.
Option 3: Calculate the sum of Stack Integers.
Option 4: Quit.
Enter the number to do corresponding task.
1
Please enter the integer you want to push.
2
Enter the number to do corresponding task.
1
Please enter the integer you want to push.
3
Enter the number to do corresponding task.
1
Please enter the integer you want to push.
1
Enter the number to do corresponding task.
2
The integer 1 on the top has been popped.
Enter the number to do corresponding task.
3
The sum of all integers on the stack is 5.
Enter the number to do corresponding task.
4
The stack has been popped.
```

Analysis:

1. Key Challenges & Solutions

- Understand the performance of the link-list-based stack and its principle of LIFO.
- Code the push, pop and sum functions based on the theory of stack.

2. Result Validation

- When implementing the pop one integer and sum function, check whether the stack is empty and

show the error message.

- Appropriate method to free all the memory of the whole stack when implementing option 4.

3. Improvement Suggestions

- Can use a method to cope with the situation that the memory has not been allocated rightly after using the *malloc()* function.
- Can optimize the name of the variables.

Conclusion:

To this lab, we need to know the theory of the link-list-based stack and its dynamic data structures.

Further, we need to know the specific performance of its push, pop and other related functions.

Otherwise, we need to learn about the right way to allocate the memory and free it.